Security Architecture

Document: `docs/architecture/SECURITY_ARCHITECTURE.md`
System: HackNest — Student Operating System
Status: Authoritative — reconciled against `README.md`, `MASTER_ARCHITECTURE.md`, and `DATABASE_ARCHITECTURE.md`
Type: Architecture Specification Only — No Implementation, No Middleware, No SQL, No API Endpoints, No Configuration

> This document follows `README.md`'s index and `MASTER_ARCHITECTURE.md`'s governance exactly. Per `MASTER_ARCHITECTURE.md` Section 9 (Cross-Document Precedence), this document is the **sole authoritative owner** of: authentication/authorization architecture, secrets management, and compliance. It explicitly does **not** own the audit log record specification, RLS policy design, or numeric RPO/RTO targets — those belong to `DATABASE_ARCHITECTURE.md` and `DEPLOYMENT_ARCHITECTURE.md` respectively, and this document cites them rather than restating them. This is a reconciliation of the standalone `SECURITY_ARCHITECTURE.md` produced earlier in this workstream, now corrected against the foundational documents that previously didn't exist.

---

0. Relationship to Governing Documents

- **Vision alignment:** security exists to make the vision in `MASTER_ARCHITECTURE.md` Section 1 trustworthy — a Student Operating System is only viable if students' data is safe in it.
- **Principle inheritance:** this document's principles (Section 1) are the security-specific refinement of the core principles in `MASTER_ARCHITECTURE.md` Section 2 (Server-first, Defense in Depth, Specify before building), not a separate, competing set.
- **Role alignment:** this document uses the canonical role set frozen in `MASTER_ARCHITECTURE.md` Section 5 exclusively: `student`,

`moderator`, `admin`, `service`. No other role name appears anywhere in this document.

- **Domain alignment:** every domain referenced below (Students, Teams, Files, Audit Logs, etc.) is the domain as specified in `DATABASE_ARCHITECTURE.md` Section 4 — this document does not redefine any domain's data shape.

---

1. Security Principles

- **Zero Trust.** No request is trusted by default regardless of origin — session state, network position, and prior authentication do not imply ongoing trust.
- **Least Privilege.** Every identity is granted the minimum access required for its function, scoped narrowly and explicitly rather than inherited broadly.
- **Defense in Depth.** Authorization is enforced independently at the database layer (RLS, owned by `DATABASE_ARCHITECTURE.md`), the API layer (`API_ARCHITECTURE.md`), and this document's own controls. No single layer is sufficient on its own.
- **Secure by Default.** New tables, routes, and features are private and inaccessible until deliberately opened up.
- **Privacy by Design.** Data collection is intentional and minimal, per `MASTER_ARCHITECTURE.md` Section 13.
- **Fail Securely.** An incomplete or ambiguous check denies the action rather than allowing it.

---

2. Threat Model

This section identifies the threat actors and attack surfaces this architecture is designed against, so that every subsequent section can be read as a specific answer to a specific threat.

Threat actors:

Actor	Motivation	Primary Surface
Unauthenticated external attacker	Data theft, account takeover, defacement, abuse of open surfaces	Public Route Handlers, authentication flows, public profile/hackathon pages
Authenticated malicious student	Access to other students' private data, team/project sabotage, harassment via Chat	Authorization boundaries between students, Chat/Messages, Team/Project membership
Compromised or malicious third-party integration	Abuse of OAuth trust, credential leakage	Google/GitHub OAuth flows, any future external integration
Insider (compromised admin/moderator credential, or a rogue elevated identity)	Broad unauthorized access, data exfiltration, cover-up via log tampering	Admin domain, Service Role, Audit Logs
Automated/bot traffic	Credential stuffing, scraping, spam, denial of service via volume	Authentication endpoints, Search, public discovery surfaces, Messages
Supply-chain / dependency compromise	Injection of malicious code via a compromised	Build pipeline, CI/CD (owned by `DEPLOYMENT_ARCHITECTURE.md`, referenced here for the security-relevant checks it must run)

Actor	Motivation	Primary Surface
	package or CI step	

Primary attack surfaces, each addressed by a specific section below: authentication (Sections 3, 9–11), authorization (Sections 4–7), the API boundary (Section 8), file uploads (Section 14), and the human/process layer (Sections 19, 20, 23).

Out of scope for this document: physical security of underlying infrastructure providers (Vercel, Supabase) is inherited, not designed by HackNest, and is noted as an accepted, monitored dependency rather than a gap this document can close.

---

3. Authentication Architecture

- **Email/Password.** Passwords are never stored or transmitted in plain form; credential verification is delegated to Supabase Auth rather than a custom implementation. Password strength requirements are enforced at creation and change time.
- **Google OAuth / GitHub OAuth.** Both are supported as federated identity providers (see Section 10). HackNest trusts each provider's assertion for authentication only, never as an implicit authorization grant.
- **Session Management.** See Section 9.
- **Token Lifecycle.** See Section 11.
- **Password Reset.** Time-boxed, single-use, token-based. A requested reset never confirms or denies whether an email address has an account, preventing account enumeration.
- **Email Verification.** New accounts are unverified until the student confirms their email. The exact list of actions gated behind verification is a product decision that traces to `MASTER_ARCHITECTURE.md`'s domain ownership rather than this document inventing one; this document asserts only that verification-gating is a required mechanism, available for any domain to use.

---

4. Authorization Architecture

Authorization is checked, at minimum, at two independent layers per the Defense in Depth principle:

1. **Database layer** — Row Level Security, whose *design* is owned by `DATABASE_ARCHITECTURE.md` Section 19; this document does not redefine it.
2. **API layer** — explicit authorization checks in every Route Handler and Server Action, per `API_ARCHITECTURE.md`, checking both role (Section 5 below) and resource ownership (per `DATABASE_ARCHITECTURE.md` Section 3's identity-ownership dimension).

Authorization decisions are never inferred from a client-supplied identifier (e.g., a `studentId` in a request body is never trusted as identity — identity comes only from the verified session, per Section 9).

---

5. Role Hierarchy

The canonical role set, frozen in `MASTER_ARCHITECTURE.md` Section 5, is:

```
service  (highest technical privilege, narrowest scope —
automated/system identities)
admin    (highest human privilege, platform-wide reach)
moderator (intermediate — content/community health, scoped
below admin)
student  (baseline — the default identity for every
registered person)
```

This is a **privilege hierarchy, not a strict inheritance hierarchy** — `admin` does not automatically gain every permission a `service` identity has (some service-only operations are intentionally not available to any human role, per Least Privilege), and `moderator` does not gain every `admin` permission scoped down; moderator permissions are a distinct, purpose-built subset (Section 6), not "admin minus some things."

Every role is assigned per-identity, and an identity may hold at most one primary role at a time (a `moderator` who becomes an `admin` is promoted, not additively granted both).

6. Permission Model

- **RBAC baseline.** Role (Section 5) determines the ceiling of what an identity can do.
- **Ownership checks.** Many actions are additionally gated by relationship to the resource (per `DATABASE_ARCHITECTURE.md` Section 3's identity ownership) — a `student` may act on their own Profile or a Project they belong to, independent of any elevated role.
- **Permission inheritance.** Scoped strictly to a resource hierarchy (e.g., a Team owner inherits broader permissions over that Team's Projects) and never propagates platform-wide.
- **Admin permissions.** Platform-wide, reserved for `admin`. Every admin action is captured by the Audit Log Record Specification owned by `DATABASE_ARCHITECTURE.md` Section 9.
- **Moderator permissions.** Scoped to content and community health (handling reports, restricting abusive accounts) — explicitly excludes billing, infrastructure, and platform configuration, which remain `admin`-only.
- **Service permissions.** Narrowly scoped per automated function (e.g., the `service` identity that computes Analytics aggregates has no reason to hold Chat moderation permissions). No `service` identity is granted blanket admin-equivalent access for convenience.

---

7. Row Level Security Governance

- **Policy design ownership:** `DATABASE_ARCHITECTURE.md` Section 19 owns RLS policy design (which domains are protected, the conceptual access rule per domain). This document does not restate that model.
- **This document's governance role:** RLS is one of two independent enforcement layers (Section 4). This document asserts the *cooperation contract* — the API layer must never assume RLS alone is sufficient, and RLS must never be treated as a substitute for explicit API-layer authorization. Neither layer is permitted to be the sole check for any sensitive operation.

- **Service Role governance:** the Supabase Service Role (which bypasses RLS) is usable only by trusted, server-side `service`-identity operations, per `DATABASE_ARCHITECTURE.md` Section 18, and is never exposed to any client context. This document adds the governance requirement that every Service Role usage site is itself subject to the Security Review Checklist (Section 24) before it is approved.

---

8. API Security

Full API contract standards are owned by `API_ARCHITECTURE.md`. This document's security-specific requirements, which every API surface must meet:

- **Authentication requirements** declared explicitly per surface; nothing is implicitly public.
- **Authorization flow** checked after authentication, per Section 4, on every request.
- **Rate limiting** per Section 17.
- **Input sanitization and validation** per Section 16.
- **Output encoding** so stored content can never be misinterpreted as executable markup by a consuming client.
- **Error handling** that is minimal and generic to the client, with full diagnostic detail retained only in server-side logs (Section 20).
- **CSRF protection.** State-changing requests relying on cookie-based session auth (Server Actions from HackNest's own frontend) are protected consistent with the framework's built-in Server Action protections. Route Handlers intended for external/token-based consumption are exempted from cookie-based CSRF concerns but must never accept cookie-based auth as a substitute for token auth.

---

9. Session Management

- Sessions are issued by Supabase Auth and validated server-side on every authenticated request; the client never determines its own authentication state for access-control purposes.

- Session cookies are HttpOnly, Secure, and scoped with an appropriate SameSite policy, per the Frontend Security controls this document governs (Section 8's CSRF note and `UI_DESIGN_SYSTEM.md`'s storage-policy alignment).
- Sessions are revocable server-side (sign-out, security-triggered force-revocation) and expire on a defined schedule (see Section 11).
- No sensitive session or credential material is ever stored in Local Storage or Session Storage, consistent with `UI_DESIGN_SYSTEM.md`'s Frontend Security-adjacent storage policy.

---

10. OAuth Security

- Google and GitHub are supported as federated identity providers. OAuth is used strictly for **authentication** (proving identity), never as an implicit **authorization** grant (e.g., linking GitHub does not itself grant any elevated permission).
- OAuth state parameters are used to prevent CSRF-style authorization-code interception during the OAuth handshake.
- Linked-provider data (e.g., GitHub profile enrichment) is only used with explicit student consent, per `MASTER_ARCHITECTURE.md` Section 13's Privacy by Design principle, and is treated as enrichment to the Student Profile domain (`DATABASE_ARCHITECTURE.md`), not a separate, competing identity source.
- OAuth client secrets are governed under Section 12 (Secrets Management) — never exposed to any client-executed code.

---

11. JWT Strategy

- Session and access tokens are issued and signed by Supabase Auth; HackNest does not implement a custom JWT signing scheme, minimizing the surface area it must independently secure.
- Access tokens are short-lived; refresh tokens are longer-lived but independently revocable, so a compromised access token has a bounded window of usefulness.

- Token validation occurs server-side on every request that requires authentication; a token's mere presence is never sufficient — it is verified against the issuing authority each time trust matters (not solely decoded and trusted client-side or cached indefinitely without re-validation on sensitive actions).
- JWT signing secrets/keys are managed entirely by the authentication provider and are never directly held or rotated by HackNest application code — see Section 12 for how this bounds HackNest's own secret-rotation responsibility.

---

12. Secrets Management

- **Environment variables.** All secrets are supplied via environment configuration specific to each deployment environment (per `DEPLOYMENT_ARCHITECTURE.md`'s Environments) and are never committed to source control.
- **API keys.** Third-party API keys are scoped to the minimum permission set required and held only in server-side environment configuration.
- **OAuth secrets.** Client secrets for Google, GitHub, and any future provider are stored exclusively server-side, never referenced from client-executed code.
- **JWT secrets.** Owned by Supabase Auth (Section 11); HackNest's responsibility is limited to correctly configuring, not independently generating or storing, these.
- **Rotation policy.** Secrets are rotated on a defined cadence and immediately upon any suspected exposure, designed with overlap windows where feasible so sessions/integrations aren't abruptly invalidated. Exact numeric rotation cadence is a placeholder pending `MASTER_GOVERNANCE_AMENDMENT_001.md`'s operational guidance, flagged rather than invented.
- **Storage rules.** Secrets are never logged, never included in error messages or Audit Log context fields (per `DATABASE_ARCHITECTURE.md` Section 9's explicit exclusion of secrets from audit context), and never exposed through any API response.

---

13. Encryption Strategy

- **In transit.** All traffic is encrypted via TLS by default across every environment, including Preview, per `DEPLOYMENT_ARCHITECTURE.md`'s SSL/TLS section, which this document defers to rather than restating certificate management mechanics.
- **At rest.** Database and object storage encryption at rest is inherited from the managed Supabase/storage infrastructure; this document's responsibility is ensuring no application-level workaround (e.g., storing sensitive data in an unencrypted side-channel) undermines that inherited guarantee.
- **Field-level encryption.** Reserved for data categories where at-rest infrastructure encryption alone is judged insufficient (e.g., highly sensitive verification tokens); applied deliberately per-domain rather than blanket, to avoid unnecessary complexity where it adds no real protection beyond what's already inherited.

---

14. File Upload Security

- **Allowed file types.** Restricted to an explicit allow-list per upload context (avatars, cover images, project media, documents, organization assets — per `DATABASE_ARCHITECTURE.md` Section 20's storage-category mapping). Anything outside the allow-list is rejected before storage.
- **Size limits.** Defined per context, appropriate to its purpose.
- **Virus scanning.** Every upload is scanned before being made available for retrieval by any other user; infected files are rejected and the attempt is captured in the Audit Log (per `DATABASE_ARCHITECTURE.md` Section 9's specification).
- **Image validation.** Validated for genuine image content, not merely a permitted extension, guarding against disguised payloads.
- **Metadata stripping.** Identifying metadata (e.g., embedded location data) is stripped before storage or distribution, per Privacy by Design.

---

15. Storage Security

- Object storage access is never granted directly to clients for write operations; all uploads flow through server-side validation (Section 14) before reaching storage.
- Read access to stored objects follows the same RLS-equivalent visibility rules as the owning domain's metadata record (per `DATABASE_ARCHITECTURE.md` Section 20) — a private Project's media is not retrievable merely by guessing or discovering its storage path.
- Storage buckets/categories are isolated per environment (Local/Preview/Staging/Production), consistent with `DEPLOYMENT_ARCHITECTURE.md`'s environment isolation principle — no environment's uploads are visible to another.

---

16. Input Validation Standards

- Every request payload (body, query, route params) is validated against a defined schema before reaching business logic, per `API_ARCHITECTURE.md`'s Request Standards, which this document's security requirement reinforces rather than duplicates in mechanism.
- Validation failures return the standard validation error format (`API_ARCHITECTURE.md`) without echoing back sensitive input values.
- All input that may be rendered, stored, or interpolated is treated as untrusted and normalized/sanitized appropriately for its eventual destination — this is the governing rule that prevents both injection-style attacks and stored XSS (Section 8).

---

17. Rate Limiting Architecture

- Every externally reachable endpoint and every abuse-sensitive internal action (authentication, password reset, invites, messaging) is rate-limited.
- **Tier ownership:** the actual numeric/tiered rate-limit definitions are owned by `API_ARCHITECTURE.md`, per `MASTER_ARCHITECTURE.md` Section 9 — this document asserts *which categories of action require*

stricter limits (authentication and password reset stricter than general reads) but defers the specific thresholds to `API_ARCHITECTURE.md`, which is flagged (per `MASTER_ARCHITECTURE.md`'s own validation) as needing a future revision to actually enumerate them rather than continuing to defer indefinitely.
- Rate limits fail closed — if the rate-limiting mechanism itself is unreachable or errors, the safer behavior is to deny or degrade rather than silently allow unlimited traffic through.

---

18. Abuse Prevention

- **Account-level abuse** (spam account creation, credential stuffing) is mitigated by rate limiting (Section 17) on authentication endpoints and by email verification (Section 3) gating meaningful platform actions.
- **Content-level abuse** (harassment in Chat/Messages, spam in Search/Discovery surfaces) is mitigated by the Moderator permission tier (Section 6) and a reporting mechanism whose data ownership belongs to the domain being reported on, with moderation action itself captured in the Audit Log.
- **Scraping/enumeration abuse** is mitigated by ensuring no endpoint returns data in a way that allows trivial sequential enumeration of private resources (consistent with UUID primary keys per `DATABASE_ARCHITECTURE.md` Section 7, which are specifically chosen to resist enumeration).
- **Automated/bot abuse** at the authentication and public-surface layer is mitigated by rate limiting plus, where warranted for the highest-risk flows (e.g., repeated failed logins), progressive friction (Section 19's failed-login monitoring feeding into this).

---

19. Audit Logging Architecture

- **Record specification ownership:** the actual shape of an audit log record (actor, action, target, timestamp, context, severity) is owned entirely by `DATABASE_ARCHITECTURE.md` Section 9 — this document

does not redefine it, correcting the three-way ownership ambiguity identified in `ARCHITECTURE_AUDIT_REPORT.md`.

- **This document's governance role:** defining *which categories of action must produce an audit record*. At minimum: authentication events (sign-in, sign-out, failed attempts, password/email changes), authorization failures, permission/role changes, every `admin` action, every Service Role invocation, and every file upload rejection (Section 14).
- Audit records are immutable once written and are never editable by any role, including `admin` — only appendable, and only viewable/exportable, consistent with their permanent-retention treatment in `DATABASE_ARCHITECTURE.md` Section 15.

---

20. Monitoring & Alerting

- **Security events** (failed authentication, authorization failures, permission escalation, account-security changes) are logged as a distinct stream from general application logs, feeding both the Audit Log (Section 19) and real-time monitoring.
- **Failed login monitoring.** Repeated failed attempts against a single account or originating from a single source trigger progressive friction (temporary lockout, step-up verification) rather than being silently absorbed.
- **Alerting.** Anomalous patterns — spikes in failed logins, unusual `admin` / `service` activity, unexpected data-export volume — trigger operator alerts rather than being visible only retroactively in logs. The underlying monitoring infrastructure itself (dashboards, uptime checks, application metrics) is owned by `DEPLOYMENT_ARCHITECTURE.md`'s Monitoring & Observability section; this document's requirement is that security-relevant signal is a distinct, explicitly monitored category within that infrastructure, not folded silently into general application health metrics.

---

21. Backup Security

- Full backup scheduling, restore process, and Recovery Point/Time Objectives are owned by `DEPLOYMENT_ARCHITECTURE.md` per `MASTER_ARCHITECTURE.md` Section 9 — not restated here.
- This document's security-specific requirement: backups are subject to the same encryption-at-rest and access-control expectations as production data itself (Section 13) — a backup is not a lower-security copy of the data it protects.
- Access to restore a backup is itself an `admin` / `service`-tier action, captured in the Audit Log per Section 19.

---

22. Privacy & Compliance

- **User privacy.** Student data is used only for disclosed purposes, per Privacy by Design (`MASTER_ARCHITECTURE.md` Section 13). Cross-domain data is not repurposed for undisclosed uses without explicit consent.
- **Data retention.** Retention categories per domain are owned by `DATABASE_ARCHITECTURE.md` Section 15; this document's compliance requirement is that deletion requests (below) are actually honored within those defined categories.
- **Account deletion.** Students can request deletion; propagation across owned/associated data follows the soft-delete-then-retention-grace-period model in `DATABASE_ARCHITECTURE.md` Sections 8 and 15. As the Architecture Audit noted, no single roadmap phase currently owns the cross-domain propagation logic this requires — this document flags that gap as still open pending a roadmap revision, rather than treating it as silently resolved.
- **Consent.** Optional data use (OAuth-based profile enrichment, marketing communication) requires explicit, revocable consent, never bundled into required account terms.
- **Security review process.** See Section 24.

---

23. Incident Response Process

- **Trigger:** any suspected unauthorized access, data exposure, or integrity violation.
- **Containment first:** the immediate priority is stopping ongoing exposure (e.g., revoking compromised sessions/credentials via Section 9's revocation capability, disabling an affected surface) before full root-cause investigation.
- **Investigation:** uses the Audit Log (Section 19) and monitoring signal (Section 20) as the primary evidence sources — this is a direct reason both must be tamper-resistant and comprehensive.
- **Disclosure:** decisions about student/regulatory notification are a governance-layer decision, formally owned by `MASTER_GOVERNANCE_AMENDMENT_001.md` once it exists; until then, this document asserts that disclosure decisions require the same interim approval authority defined in `MASTER_ARCHITECTURE.md` Section 5, not an individual engineer's unilateral judgment.
- **Remediation:** the specific vulnerability or gap is closed, and — critically — this document, `DATABASE_ARCHITECTURE.md`, or the relevant subsystem document is updated if the incident revealed a specification gap, not just a code-level bug, closing the loop back into the documentation set per `MASTER_ARCHITECTURE.md` Section 11's Change Management process.
- **Roles and escalation paths** are defined ahead of time, not improvised during an incident — full staffing/escalation detail is an `OPERATIONS_ARCHITECTURE.md` concern (next in the remediation sequence), which this document defers to for the operational mechanics while retaining ownership of the security-decision authority described above.

---

24. Security Review Checklist

Required before any change touching authentication, authorization, sensitive data, Service Role usage, or file handling is considered ready for release:

- [ ] Authentication/session behavior verified against Sections 3, 9, 11.

- [ ] Authorization checked at both API and RLS layers independently (Section 4, 7).
- [ ] Role/permission scope verified against Sections 5–6 (no unintended privilege escalation).
- [ ] Input validation and output encoding verified (Sections 8, 16).
- [ ] Rate limiting applied to any new or changed endpoint per its category (Section 17).
- [ ] Any new upload path validated against Sections 14–15.
- [ ] Any new sensitive action confirmed to produce an Audit Log record per the categories in Section 19.
- [ ] No secret or credential material introduced outside the rules in Section 12.
- [ ] Privacy/consent implications reviewed against Section 22 for any new data collection.

Definition of Done

- [x] Architecture only — no implementation code, middleware, SQL, API endpoints, or configuration files.
- [x] Follows `README.md` and `MASTER_ARCHITECTURE.md` exactly.
- [x] All 24 required topics covered.
- [x] Cross-referenced against `DATABASE_ARCHITECTURE.md` without restating its owned content.

Validation Report

Internal Consistency

Every section that touches a concern owned by another document (RLS policy design, audit log record shape, rate-limit tiers, RPO/RTO) explicitly defers rather than restates, and every section this document itself owns (authentication, authorization architecture, secrets, compliance) is defined exactly once and referenced consistently elsewhere in the document (e.g.,

Section 19's audit-trigger categories reference, but do not redefine, Section 9's record spec). No internal contradiction found.

Consistency with `README.md`, `MASTER_ARCHITECTURE.md`, `DATABASE_ARCHITECTURE.md`

- Uses the canonical role set from `MASTER_ARCHITECTURE.md` Section 5 exclusively (Section 5 above).
- Defers RLS policy design to `DATABASE_ARCHITECTURE.md` Section 19, RPO/RTO to `DEPLOYMENT_ARCHITECTURE.md`, and rate-limit tiers to `API_ARCHITECTURE.md`, per `MASTER_ARCHITECTURE.md` Section 9's ownership table.
- Cites the Audit Log Record Specification from `DATABASE_ARCHITECTURE.md` Section 9 directly (Section 19 above) instead of independently defining it, correcting the prior standalone draft's overlap.
- Confirmed consistent with all three governing documents.

Dependency Check

Dependency	Status
`README.md`	Available and followed.
`MASTER_ARCHITECTURE.md`	Available and followed (roles, precedence, principles).
`DATABASE_ARCHITECTURE.md`	Available and followed (audit log spec, RLS ownership, storage/file domain mapping).
`API_ARCHITECTURE.md`	Available; this document defers rate-limit tiers to it, and flags that document still needs revision to actually enumerate them.

Dependency	Status
`DEPLOYMENT_ARCHITECTURE.md`	Available; this document defers RPO/RTO, TLS/certificate mechanics, and monitoring infrastructure to it.
`MASTER_GOVERNANCE_AMENDMENT_001.md`	**Still missing.** Secret-rotation cadence, disclosure-decision authority, and formal security-review sign-off authority remain flagged placeholders pending it.
`OPERATIONS_ARCHITECTURE.md`	**Not yet produced** (next in remediation sequence). Incident-response staffing/escalation mechanics are deferred to it.

Risk Assessment

Area	Residual Risk	Rationale
Rate-limit tiers undefined in numeric terms	**Medium**	Category-level guidance exists here, but `API_ARCHITECTURE.md` has not yet been revised to enumerate actual thresholds.
Account-deletion cross-domain propagation ownership	**Medium**	Flagged as an open roadmap gap, not newly introduced by this document, but not yet closed either.
Incident disclosure authority	**Medium**	Interim authority is asserted (per

Area	Residual Risk	Rationale
		`MASTER_ARCHITECTURE.md` Section 5) but not formally constituted pending governance document.
Secret rotation cadence	**Low-Medium**	Principle and triggers are defined; exact cadence is a placeholder.
Core authentication/authorization/RLS-cooperation model	**Low**	Fully specified and consistent with `DATABASE_ARCHITECTURE.md`; no open gap identified.

Confirmation

This document is internally consistent, consistent with `README.md`, `MASTER_ARCHITECTURE.md`, and `DATABASE_ARCHITECTURE.md`, and resolves every security-related finding raised in `ARCHITECTURE_AUDIT_REPORT.md` that was within its own ownership to resolve (the audit-log ownership ambiguity is fully closed; the rate-limit-tier gap and RPO/RTO duplication are correctly redirected to their true owning documents rather than re-duplicated here).

Status: Validated — Ready to Govern All Implementation Phases